exec

April 14, 2025

An Exception was encountered at ‘In [1]’.

Execution using papermill encountered an exception here and stopped:

[1]:
```
import numpy as np
import torch
import tensorly as tl
from tensorly.decomposition import parafac
import sparse
import pickle
import os

assert os.path.exists('sptensor.pkl'), 'No such file.'
with open('sptensor.pkl', 'rb') as f:
    data = pickle.load(f)
data = data[:, :, :, :12, :]
expected_shape = data.shape
n_components = 10

# --------------------------------
# 3. NumPy-based BPTF (Aaron's original implementation)
# --------------------------------
# Import Aaron's BPTF (which uses NumPy/sparse.COO);
# ensure that the bptf package is in your PYTHONPATH.
from bptf import BPTF as BPTF
import bptf

# Create the same data as a NumPy array and a corresponding binary mask
data_np = sparse.COO(data.copy())
mask_np = np.ones(expected_shape, dtype=int)
mask_np = sparse.COO(mask_np.copy())

# Instantiate and fit the NumPy-based BPTF model.
# Note: This version uses its own preprocess() function and can work with␣
 ↪sparse.COO.
model_np = BPTF(data_shape=data_np.shape, n_components=n_components)
model_np.fit(data, mask=mask_np, max_iter=50, verbose=True)
```

```
# Reconstruct using arithmetic expectation
reconstruction_np = model_np.reconstruct(mask=None, fill_value=0,␣
 ↪drop_diag=False, style='arithmetic')
frobenius_diff_np = np.sqrt(np.sum((data.todense() - reconstruction_np)**2))
print("NumPy BPTF reconstruction Frobenius norm difference:", frobenius_diff_np)

# --------------------------------
# 1. PyTorch-based BPTF (your version)
# --------------------------------
# Import your PyTorch-based BPTF model (adjust filename as needed)
from own_implementation import BPTF as BPTF_torch
tl.set_backend('pytorch')

device = 'cpu'
# Create a tensor from a Poisson distribution (counts) and a matching mask;␣
 ↪ensure types match
data_torch = torch.tensor(data.todense(), dtype=torch.float64, device=device)
mask_torch = torch.ones(expected_shape, dtype=torch.float64, device=device)

# Instantiate and fit the PyTorch-based BPTF model
model_torch = BPTF_torch(data_shape=expected_shape, n_components=n_components,␣
 ↪device=device)
model_torch.fit(data_torch, mask=mask_torch, max_iter=50, tol=1e-4,␣
 ↪verbose=True)
reconstruction_torch = model_torch.reconstruct(mask=mask_torch,␣
 ↪style='arithmetic')
frobenius_diff_torch = torch.norm(data_torch - reconstruction_torch, p='fro').
 ↪item()
print("PyTorch BPTF reconstruction Frobenius norm difference:",␣
 ↪frobenius_diff_torch)

# --------------------------------
# 2. TensorLy CP Decomposition
# --------------------------------
# Use TensorLy's parafac for CP decomposition (same rank as n_components)
cp_decomp = parafac(data_torch, rank=n_components, n_iter_max=100,␣
 ↪init='random')
reconstruction_cp = tl.cp_to_tensor(cp_decomp)
frobenius_diff_cp = torch.norm(data_torch - reconstruction_cp, p='fro').item()
print("TensorLy CP decomposition Frobenius norm difference:", frobenius_diff_cp)

# def _check_mode(self, m):
#     assert np.isfinite(np.asarray(self.E_DK_M[m])).all()
#     assert np.isfinite(np.asarray(self.G_DK_M[m])).all()
#     assert np.isfinite(np.asarray(self.shp_DK_M[m])).all()
#     assert np.isfinite(np.asarray(self.rte_DK_M[m])).all()
```

```
# bptf.BPTF._check_mode = _check_mode
```

```
ITERATION 0:    Time: 0.000000  Objective: 18413338.42  Change: nan


  0%|
| 0/50 [00:00<?, ?it/s]

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 0]  min uttkrp_DK = -3.725e-09
[mode 0]  min new rate = 9.899e-02   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 1]  min uttkrp_DK = -9.441e-05
[mode 1]  min new rate = 9.898e-02   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

/home/luiyusen/projects/bptf_new/bptf/src/bptf/bptf.py:218: RuntimeWarning:
invalid value encountered in log
  self.G_DK_M[m] = np.exp(sp.psi(shp_DK) - np.log(rte_DK))


  0%|
| 0/50 [00:23<?, ?it/s]

Are the shape parameters positive? True
Are the rate parameters positive? False
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 2]  min uttkrp_DK = -7.900e+01
[mode 2]  min new rate = -7.890e+01   negatives = 120
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? False
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
```

```
Cell In[1], line 32
     29 # Instantiate and fit the NumPy-based BPTF model.
     30 # Note: This version uses its own preprocess() function and can work␣
 ↪with sparse.COO.
     31 model_np = BPTF(data_shape=data_np.shape, n_components=n_components)
---> 32 model_np.fit(data, mask=mask_np, max_iter=50, verbose=True)
     34 # Reconstruct using arithmetic expectation
     35 reconstruction_np = model_np.reconstruct(mask=None, fill_value=0,␣
 ↪drop_diag=False, style='arithmetic')

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:289, in BPTF.fit(self, data,␣
 ↪mask, missing_val, **kwargs)
    285     assert is_binary(mask)
    287 self._init()
--> 289 self._update(data, mask=mask, missing_val=missing_val, **kwargs)
    290 return self

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:252, in BPTF._update(self, data,
 ↪mask, modes, **kwargs)
    250     self._update_variational_params(m, data, mask)
    251     self._update_beta(m)
--> 252     self._check_mode(m)
    253 bound = self._elbo(data, mask=mask)
    254 delta = (bound - curr_elbo) / abs(curr_elbo)

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:159, in BPTF._check_mode(self, m
    157 def _check_mode(self, m):
    158     assert np.isfinite(self.E_DK_M[m]).all()
--> 159     assert np.isfinite(self.G_DK_M[m]).all()
    160     assert np.isfinite(self.shp_DK_M[m]).all()
    161     assert np.isfinite(self.rte_DK_M[m]).all()

AssertionError:
```